

虚函数

1. 一个类只有一个虚函数表。
2. 实现多态的函数
 - [1. 类型相容](#)
 - [2. 绑定时间](#)
 - [2.1. 前期绑定\(Early Binding\)\(静态绑定\)](#)
 - [2.2. 动态绑定\(Late Binding\)](#)
 - [2.3. 后期绑定的实现](#)
 - [2.4. 注重效率](#)
 - [3. 定义](#)
 - [4. 虚函数限制](#)
 - [5. final, override](#)
 - [6. 纯虚函数和抽象类](#)
 - [6.1. 纯虚函数\(Java中的接口\)](#)
 - [6.1.1. 抽象类](#)
 - [6.2. 示例一:Figure](#)
 - [6.3. 抽象工厂模式](#)
 - [7. 虚析构函数](#)
 - [8. 继承的分类](#)
 - [8.1. public inheritance](#)
 - [8.1.1. Penguin问题](#)
 - [8.1.2. 如何实现里氏替换原则](#)
 - [8.1.3. 长方形问题](#)
 - [8.1.4. 不要定义与继承而来的非虚成员函数同名的成员函数](#)
 - [8.2. private inheritance 私有继承](#)
 - [8.3. 和组合的联系](#)
 - [8.3.1. 情况一](#)
 - [8.3.2. 情况二](#)
 - [8.3.3. Example](#)
 - [9. 虚函数的分类](#)
 - [9.1. 纯虚函数](#)
 - [9.2. 一般虚函数](#)
 - [9.3. 非虚函数](#)
 - [10. 绝对不要重新定义继承而来的缺省参数值!](#)
 - [10.1. Example](#)
 - [11. 参考](#)

1. 类型相容

1. 类、类型:

2. 类型相容:

1. 类型相容是指完全相同的(别名)
2. 一个类型是另一个类型的子类型(int -> long int)

3. 赋值相容(不会丢失信息):对于类型相同的变量才有

1. 如果类型相同可以直接赋值
2. 子类型可以赋值给父类型

4. 问题:a和b都是类, a、b什么类型时, a = b合法(赋值相容)? B是A的子类型的时候

- `A a; B b; class B: public A`
 - 对象的身份发生变化(a和b都代表栈上对应大小的内存),B类型对象变为了A类型的对象
 - 属于派生类的属性已不存在
 - 将派生类对象赋值给基类对象->对象切片
- `A a = b;`调用拷贝构造函数
- `const A &a;`函数必然包含的拷贝构造函数中的参数
- `B* pb; A* pa = pb; class B: public A`
 - 因为是赋值相容的, 所以可以指针赋值
 - 这种情况类似java
- `B b; A &a=b; class B: public A`: 对象身份没有发生变化(还是B)

5. 把派生类对象赋值给基类对象, 基类的引用或指针可以引用或指向派生类对象, 不严谨的说, 可以说让父类指向子类

6. 传参的时候尽量不要拷贝传参(存在对象切片问题), 而是使用引用传参。

```
1 //测试切片调用
2 class A{
3     int x,y;
4     public:
5         void f();
6 };
7 class B: public A{
8     int z;
9     public:
10        void f();
11        void g();
12 };
13 //把派生类对象赋值给基类对象
14 A a;
15 B b;
16 a = b;      //OK,
17 b = a;      //Error
18 a.f();      //A::f()
19
20 //基类的引用或指针可以引用或指向派生类对象
21 A &r_a = b;   //OK
22 A *p_a = &b; //OK
```

```

23
24 B &r_b = a;      //Error
25 B *p_b = &a;    //Error
26 //以下两个部分基本是一致的
27 func1(A& a){a.f();}
28 func2(A *pa){pa->f();}
29 func1(b); //A::f
30 func2(&b);

```

1. func1(b):为什么是A的呢?

1. 对于B, A的版本的对应函数被隐藏
2. 静态绑定是只看形参类型

2. 绑定时间

1. C++默认静态绑定

2.1. 前期绑定(Early Binding)(静态绑定)

1. 编译时刻确定调用哪一个方法
2. 依据对象的静态类型
3. 效率高、灵活性差
4. 静态绑定根据形参决定

2.2. 动态绑定(Late Binding)

1. 晚绑定是指编译器或者解释器在运行前不知道对象的类型, 使用晚绑定, 无需检查对象的类型, 只需要检查对象是否支持特性和方法即可。
2. c++中晚绑定常常发生在使用 `virtual` 声明成员函数
3. 运行时刻确定, 依据对象的实际类型(动态)
4. 灵活性高、**效率低**
5. 动态绑定函数也就是虚函数。
6. 直到构造函数返回之后, 对象方可正常使用
7. C++默认的都是静态绑定, Java默认的都是动态绑定

2.3. 后期绑定的实现

```

1  class A{
2      int x,y;
3      public:
4          virtual f();
5          virtual g();
6          h(); //h函数是默认的
7  };
8  class B: public A{
9      int z;
10     public:
11         f();
12         h();

```

```

13 };
14 A a; B b;
15 A *p;
16 //调用情况见图

```



- $p \rightarrow f()$: 需要寻找a和b中的f()函数地址
- 如果不能明确虚函数个数, 没有办法索引
- 虚函数表(索引表, vtable): 大小可变
 - 首先构造基类的虚函数表
 - 然后对派生类中的函数, 如果查找了, 则会覆盖对应函数来生成虚函数表
- 对象内存空间中含有指针指向虚函数表
- `(**((char *)p - 4))(p) : f`的函数调用(从虚函数表拿数据), p是参数this
- 空间上和时间上都付出了代价
 - 空间: 存储虚函数表指针和虚函数表
 - 时间: 需要通过虚函数表查找对应函数地址, 多调用

```

1  class A{
2      public:
3          A() { f(); }
4          virtual void f();
5          void g();
6          void h(){
7              f();
8              g();
9          }
10 };
11 class B: public A
12 {   public:
13     void f();
14     void g();
15 };
16 //直到构造函数返回之后, 对象方可正常使用
17 //函数调用顺序, 重要考试题, 依据虚函数表
18 B b;      // A::A(), A::f, B::B()
19 //为什么调用A的f而不是B的? 因为名空间以及B没有构造。
20 A *p= &b;
21 p->f();    //B::f
22 p->g();    //A::g, g是静态绑定
23 p->h();    //A::h, B::f, A::g

```

- 尽量不要在构造函数中调用虚函数
- 此时的虚函数就是和构造函数名空间相同
- h()函数是非虚接口
 - 有不同的实现: 调用了虚函数和非虚函数
 - 可以替换部分的实现
 - 可以使得非虚函数具有虚函数的特性(让全局函数具有多态: 将全局函数做成非虚接口)

```

1  class A{
2      public:
3          virtual void f() ;
4          void g() ;
5  };
6  class B: public A{
7      public:
8          void f(B* const this) { g(); } //this g() this->g();
9          void g() ;
10 };
11 B b;
12 A* p = &b;
13 p->f(); //B::f, b.B::g

```

- g()是静态绑定
- 虚函数中调用非虚函数:所有版本是和虚函数**一致**的
- 非虚函数调用虚函数:正常
- 虚函数要严格查表，非虚函数静态确定，对应p->h()
- 注意每一个函数在调用的时候都会传入一个const的this指针

2.4. 注重效率

1. 默认前期绑定
2. 后期绑定需显式指出 virtual

3. 定义

1. 虚函数是指一个类中你希望重载的成员函数，但你使用一个基类指针或引用指向一个继承类对象的时候，调用一个虚函数时，实际调用的就是继承类的版本。

```

1  class A{
2      public:
3          virtual void f();
4  };

```

1. 定义绑定:根据实际引用和指向的对象类型
2. 方法重定义
3. 注意:如基类中被定义为虚成员函数，则派生类中对其重定义的成员函数均为虚函数，也就是派生类中的对应函数可以不写虚函数。

```

1  #include <iostream>
2  using namespace std;
3
4  class Parent{
5      public:
6          char data[20];
7          void Function1();
8          virtual void Function2();    // 这里声明Function2是虚函数
9  } parent;
10 void Parent::Function1(){

```

```

11     printf("This is parent,function1\n");
12 }
13 void Parent::Function2(){
14     printf("This is parent,function2\n");
15 }
16 class Child:public Parent{
17     void Function1();
18     void Function2();
19 } child;
20 void Child::Function1(){
21     printf("This is child,function1\n");
22 }
23 void Child::Function2(){
24     printf("This is child,function2\n");
25 }
26 int main(int argc, char* argv[]){
27     Parent *p;           // 定义一个基类指针
28     if(_getch()=='c')     // 如果输入一个小写字母c
29         p=&child;         // 指向继承类对象
30     else
31         p=&parent;         // 否则指向基类对象
32     p->Function1();        // 这里在编译时会直接给出Parent::Function1()的入口地址。
33     p->Function2();        // 注意这里，执行的是哪一个Function2?
34     return 0;
35 }
36 //输入c，输出：
37 //This is parent,function1
38 //This is child,function2
39 //输入非c，输出：
40 //This is parent,function1
41 //This is parent,function2

```

4. 虚函数限制

1. 类的成员函数才可以是虚函数:全局函数不可以是虚函数
2. 静态成员函数不能是虚函数:静态的成员函数属于类，并不属于一个对象，所以不能虚函数
3. 内联成员函数不能是虚函数:内联成员函数在编译的时候就已经确定了
4. 构造函数不能是虚函数:
 1. 因为创建类的时候是自动调用的，父类的指针无法直接调用，虚函数没有意义
 2. 虚函数表是在构造函数中完成的
5. 析构函数可以(往往)是虚函数
 1. 如果不是虚函数，不好调用到派生类中的析构函数(delete一个父类指针，如果非虚，不能调用到派生类的析构函数)

5. final,override

```
1 struct B {
2     virtual void f1(int) const ;
3     virtual void f2();
4     void f3() ;
5     virtual void f5(int) final;
6 };
7 struct D: B { //继承
8     void f1(int) const override ; //正确: f1与基类中的f1 匹配
9     void f2(int) override ;      //错误: B没有形如f2(int) 的函数。
10    void f3() override ;          //错误: f3不是虚函数
11    void f4() override ;          //错误: B没有名为f4的函数
12    void f5(int) ;                //错误: B已经将f5声明成final
13 };
```

- override:希望以虚函数的形式写:编译器报错,防止漏写virtual问题
- final:不可以再次重写

6. 纯虚函数和抽象类

6.1. 纯虚函数(Java中的接口)

1. 声明时在函数原型后面加上 = 0: `virtual int f() = 0;`
2. 往往只给出函数声明,不给出实现:可以给出实现,通过函数外进行定义(但是不好访问,因为查到是0)

```
1 int f() = 0;
2 int f(){
3     Base::f; //显式调用基类中纯虚函数的定义
4 }
```

6.1.1. 抽象类

1. 至少包含一个纯虚函数
2. 不能用于创建对象:抽象类类似一个接口,提供一个框架
3. 为派生类提供框架,派生类提供抽象基类的所有成员函数的实现

```
1 class AbstractClass {
2     public:
3         virtual int f() = 0;
4 };
```

6.2. 示例一:Figure



```

1 Figure *a[100]; //Figure基类, 不会被创建出来
2 a[0] = new Rectangle();
3 a[1] = new Ellipse();
4 a[2] = new Line();
5 for (int i=0; i<num_of_figures; i++){
6     a[i]->display();
7 }

```

- 基于接口的复用，而不是基于实现的复用

6.3. 抽象工厂模式



- Step1:提供Windows GUI类库: WinButton

```

1 WinButton *pb= new WinButton();
2 pb->SetStyle();
3 WinLabel *pl = new WinLabel();
4 pl->SetText();

```

- step2:增加对Mac的支持:MacButton, MacLabel

```

1 MacButton *pb= new MacButton();
2 pb->SetStyle();
3 MacLabel *pl = new MacLabel();
4 pl->SetText();

```

- step3:增加用户跨平台设计的支持
 - 将Button抽象出来

```

1 Button *pb= new MacButton();
2 pb->SetStyle();
3 Label *pl = new MacLabel();
4 pl->SetText();
5 //创建工厂来保证创建的正确性
6 class AbstractFactory {
7 public:
8     virtual Button* CreateButton() =0;
9     virtual Label* CreateLabel() =0;
10 };
11 class MacFactory: public AbstractFactory {
12 public:
13     MacButton* CreateButton() { return new MacButton; }
14     MacLabel* CreateLabel() { return new MacLabel; }
15 };
16 class WinFactory: public AbstractFactory {
17 public:
18     WinButton* CreateButton() { return new WinButton; }
19     WinLabel* CreateLabel() { return new WinLabel; }
20 };
21 class Button; // Abstract class
22 class MacButton: public Button {};

```



```

23 class WinButton: public Button {};
24 class Label; // Abstract class
25 class MacLabel: public Label {};
26 class WinLabel: public Label {};
27 AbstractFactory* fac;
28 switch (style) {
29 case MAC:
30     fac = new MacFactory;
31     break;
32 case WIN:
33     fac = new WinFactory;
34     break;
35 }
36 Button* button = fac->CreateButton();
37 Label* Label = fac->CreateLabel();

```

- 抽象工厂模式的类图



7. 虚析构造函数

```

1 class B {...};
2 class D: public B {...};
3 B* p = new D;
4 delete p; //如果B和D的属性是相同的，则没有什么问题
5
6 class mystring {...}
7 class B {...}
8 class D: public B{
9     mystring name;
10    ...
11 }
12 B* p = new D;
13 delete p; //有问题，因为D中比B多一个mystring
14 //如果析构函数不是虚函数，则没有办法正确调用D的析构函数
15 //调用派生类析构函数 -> 调用成员对象的析构函数 -> 基类的析构函数

```

1. 如果有继承的话，最好使用虚析构函数，在调用析构的函数，会**先**调用基类的析构函数，所以：
2. 在析构函数中，只需要析构派生类自己的资源就可以了

8. 继承的分类

8.1. public inheritance

1. 确定public inheritance,是真正意义的"IS_A"关系：派生类就是基类
2. Require No more, Promiss No Less(LSP)里氏替换原则:里氏替换原则没有给出实现
3. **不要定义与继承而来的非虚成员函数同名的成员函数**

8.1.1. Penguin问题

1. 软件外包的时候不知道是谁写的，无法修改客户代码

```
1 //Penguin问题:详见软工二
2 class FlyingBird;
3 class NonFlyingBird;
4 virtual void fly() { error("Penguins can't fly!"); }
```

8.1.2. 如何实现里氏替换原则

1. Design By contract:每个方法在调用前，应该校验传入参数的正确性，正确才能进行调用，否则是违反契约的。
2. 校验前置条件:对象对自己进行校验

8.1.3. 长方形问题

```
1 //v1
2 class Rectangle {
3     public:
4         void setHeight(int);
5         void setWidth(int);
6         int height() const; //不修改的声明为const方便常对象调用
7         int width() const;
8 };
9 assert(s.width() == s.height());
10 class Square: public Rectangle {
11     public:
12         void setLength (int);
13     private: //设置为private可以避免父类的方法被单独调用
14         void setHeight(int);
15         void setWidth(int); //可以单独改变一条边
16 };
17 assert(s.width() == s.height()); //前后校验不变式
18 //检查里氏替换原则
19 Square s(1,1);
20 Rectangle *p = &s;
21 p->setHeight(10);
22 //v2 添加虚函数声明
23 class Rectangle {
24     public:
25         virtual void setHeight(int);
26         virtual void setWidth(int);
27         int height() const;
28         int width() const;
29 };
30 assert(s.width() == s.height());
31 class Square: public Rectangle {
32     public:
33         void setLength (int);
34     public: //注意是public的
35         void setHeight(int);
36         void setWidth(int); //可以单独改变一条边
37 };
```

```

38 //问题
39 //如下的操作如果传入了正方形(对于使用了这一套继承体系的代码)
40 void widen(Rectangle& r, int w)
41 {
42     int oldHeight = r.height();
43     r.setWidth(r.width() + w); //set被重写过
44     assert(r.height() == oldHeight);
45 }
46 //v3
47 class Rectangle {
48     public:
49         virtual void setHeight(int);
50         virtual void setWidth(int);
51         int height() const;
52         int width() const;};
53 assert(s.width() == s.height());
54 class Square: public Rectangle {
55     public:
56         void setLength (int);
57     private://修改为private
58         //在编译的时候决定, 确定函数位置
59         void setHeight(int );
60         void setWidth(int );//可以单独改变一条边
61 };
62 assert(s.width() == s.height());
63 void widen(Rectangle& r, int w)
64 {
65     int oldHeight = r.height();
66     r.setWidth(r.width() + w);
67     //我们直接修改对应方法为private, 这样子调用会直接报错
68     //问题:访问控制是编译的时候进行处理的, 之后运行的时候该怎么调用就怎么调用
69     //public和private的访问控制是编译时确定的, 而不是虚函数运行时决定
70     //编译的时候检查rectangle是public的, 没有问题, 通过。之后调用的时候发现是虚函数, 然
    后从虚函数表能找到private的函数, 并且调用。
71     assert(r.height() == oldHeight);
72 }

```

- 访问控制仅仅是检查这个成员时是否属于当前对象声明的静态部分中可访问, 和实际运行时时刻无关。
- 只要通过编译检查, 对于public和private没有任何问题
- 最终的问题:想要满足里氏替换原则, 衍生类的条件应该比基类更加弱, 而这个是更强

8.1.4. 不要定义与继承而来的非虚成员函数同名的成员函数

```
1  class B {
2      public:
3          void mf();
4  };
5  class D: public B {
6      public:
7          void mf();
8  };
9  D x;
10 B* pB = &x;
11 pB->mf(); //B:mf
12 D* pD = &x;
13 pD->mf(); //D:mf
```

1. 对于一个对象，在不同的指针的情况下会表示出来不同的行为，这是很麻烦的。
2. 运行时不要出现不一致的

8.2. private inheritance 私有继承

1. 特别含义:Implemented-in-term-of 用继承来实现某个功能
 - 需要使用Base Class中的protected成员，或重载virtual function
 - 表示不希望一个Base Class被client使用，否则会使用公有继承
 - 利用一些已经存在的代码，只是在实现中使用到了基类
2. 在设计层面无意义，只用于**实现**层面，只是复用实现的方式，接口是被忽略的
3. 实际上是Has-A关系
4. 如果两个类的继承是私有的，则不能在派生类外将派生类转换成基类对象。

8.3. 和组合的联系

1. 尽可能用组合，万不得已用继承

8.3.1. 情况一

需要使用protected和重载virtual function

8.3.2. 情况二

1. 我们需要尽可能复用，并且减少内存的占用。
2. 如果没有静态成员、虚函数、虚基类，那么一个类是被认为是一个空的类，而方法是定义在代码区。对象不占用空间。
3. 技术上:所有的独立对象必须有一个大小，不然会导致两个指针指向一个地址。
4. Example:算法的类图，如果使用组合，依据会生成生成一个指针

8.3.3. Example

1. 目的是为了重定义eat函数
2. 私有继承不能将派生类转换成基类对象
3. 可以强制转换(提供方法)

```
1 class CHumanBeing { ... };
2 class CStudent:private CHumanBeing { ... };
3 void eat(const CHumanBeing & h) { ... }
4 CHumanBeing a;CStudent b;
5 eat(a);
6 eat(b);//Error
```

9. 虚函数的分类

```
1 class Shape {
2     public:
3         virtual void draw() const = 0;
4         virtual void error(const string& msg);
5         int objectID() const;
6 };
```

9.1. 纯虚函数

1. 只有函数接口会被继承
 - 子类**必须**继承函数接口
 - (必须)提供实现代码

9.2. 一般虚函数

1. 函数的接口及缺省实现代码都会被继承
 - 子类**必须**继承函数接口
 - **可以**继承缺省实现代码
 - 不希望代码被经常修改，微小修改

9.3. 非虚函数

1. 函数的接口和其实现代码都会被继承
 - **必须**同时继承接口和实现代码

10. 绝对不要重新定义继承而来的缺省参数值！

1. 静态绑定
2. 效率

10.1. Example

```
1  class A{
2      public:
3          virtual void f(int x = 0) =0;
4  };
5  class B: public A{
6      public:
7          virtual void f(int x = 1)
8              { cout << x;}
9  };
10 A *p_a;
11 B b;
12 p_a = &b;
13 p_a->f();//0
14
15 class C: public A{
16     public:
17         virtual void f(int x) { cout<< x;}
18 };
19 A *p_a1;
20 C c;
21 p_a1 = &c;
22 p_a1->f();//0
23 //对象中只记录虚函数的入口地址
```

- 对象中只记录了虚函数入口的位置
- 默认参数在编译的时候静态绑定(最开始就将x绑定到所有的此类函数中去。)
- 并没有为每一个版本存储记录一个缺省值:因为如果有的话,需要用额外的空间来存储缺省值。
- 如果给出参数,则进行压栈(1),而往往有时候没有,这时候将默认参数存储虚函数表中。这会影响语言的效率。(避免寻址)

```
1  ((*((char *)p_a1 - 4))(p_a1)
2  char *q = *((char *)p_a1 - 4);
3  (*q)(p_a1, *q+4);
```



11. 参考

1. [C++中的virtual\(虚函数\)的用法](#)